AMRITA VISHWA VIDYAPEETHAM

Drone Delivery Management System

~A Database-Driven Platform for Efficient Autonomous Package Delivery

22AIE303 - Database Management Systems

Team C4:

- CB.SC.U4AIE24203 - Advaith Krishna
- CB.SC.U4AIE24204 - Amudala Jashwanth
- CB.SC.U4AIE24209 - Bhuvan Rajasekar
- CB.SC.U4AIE24251 - Sachcith G N

Introduction & Problem Statement & Solution



Introduction

- Drone delivery systems involve managing users, packages, drones, routes, batteries, and payments.
- A centralised database connects these components and maintains organized records.

Problem Statement

- Managing delivery information separately causes data redundancy and inconsistencies.
- Tracking packages, drone availability, and delivery records becomes difficult.

Proposed Solution

Develop a **Database-Driven Drone Delivery Management System** that stores and manages delivery operations through a unified relational database.

Functionalities and Associated DBMS Concepts

AMRITA VISHWA VIDYAPEETHAM

1. User and Vendor Management

Functionalities: Registration, authentication, profile management and CRUD operations.

Concepts: Relational Schema, Primary/Foreign Keys, UNIQUE Constraints, SQL DML.

2. Package and Route Management

Functionalities: Package booking, tracking, route assignment and status management.

Concepts: E–R Modelling, Referential Integrity, Relationships, SQL Queries.

3. Drone, Battery & Maintenance

Functionalities: Payload validation, battery health monitoring, charging and maintenance checks.

Concepts: Constraints, Triggers, Conditional Queries, Data Integrity.

4. Mission Creation & Resource Allocation

Functionalities: Validates package ownership, drone capacity and pilot availability before allocation.

Concepts: Stored Procedures, Transactions, Atomicity, Integrity Constraints.

Functionalities and Associated DBMS Concepts

AMRITA VISHWA VIDYAPEETHAM

5. Mission Execution and Lifecycle Management

Functionalities: Initiates, completes or cancels missions with controlled status transitions.

Concepts: Stored Procedures, BEFORE UPDATE Triggers, Atomic Multi-table Updates.

6. Concurrency and Transaction Management

Functionalities: Prevents conflicting assignments and coordinates consistent mission updates.

Concepts: Row-Level Locking (FOR UPDATE), Isolation, Commit/Rollback, ACID Properties.

7. Payment and Delivery Records

Functionalities: Maintains payment details, delivery history and linked operational records.

Concepts: Relational Mapping, Foreign Keys, CRUD Operations, Data Consistency.

Modules & Explanations - Overview

AMRITA VISHWA VIDYAPEETHAM

1. User & Vendor Management

- Handles registration, authentication, profile updates and role-based access.
- Maintains user and vendor records through CRUD operations and relational constraints.

2. Package & Route Management

- Manages package booking, delivery status, route allocation and tracking.
- Links packages, vendors, customers and routes through relational relationships.

3. Drone, Pilot & Battery Management

- Monitors drone payload capacity, pilot availability, battery health and maintenance status.
- Validates resource eligibility before mission allocation and dispatch.

4. Mission Management

- Creates, starts, completes and cancels missions using stored procedures.
- Enforces valid mission transitions through database triggers.

5. Transaction & Concurrency Management

- Performs atomic updates across missions, pilots, drones and packages.
- Uses row-level locking (SELECT ... FOR UPDATE) to coordinate concurrent dispatch operations.

6. Payment & Delivery Records

- Maintains payment details, delivery history and completed mission records.
- Preserves relationships and data consistency using foreign-key constraints.

Modules & Explanations - Stored Procedures

AMRITA VISHWA VIDYAPEETHAM

1. sp_create_mission

- Validates vendor package ownership, cargo weight against payload capacity, battery health, and maintenance status.
- Reserves the parcel and inserts an IDLE mission record within an isolated transaction block.

2. sp_start_mission

- Acquires exclusive row-level locks (SELECT ... FOR UPDATE) to prevent concurrent double-dispatch races.
- Synchronously transitions the mission to ONGOING and the assigned pilot to ON_MISSION.

3. sp_complete_mission

- Finalizes delivery by transitioning the mission to OVER and recording the exact completion timestamp.
- Atomically frees the pilot to AVAILABLE, updates drone location to the destination, and stamps LAST_DELIVERY.

4. sp_cancel_mission

- Safely aborts an unlaunched (IDLE) mission, setting its status to CANCELLED and recording cancellation time.
- Releases reserved parcels and assigned flight crew back to the available pool without deleting audit records.

5. sp_process_payment

- Verifies customer package ownership and ensures the delivery mission has reached the completed (OVER) state.
- Derives deterministic billing charges, blocks duplicate payments, and inserts the payment record atomically.

6. sp_record_maintenance

- Logs diagnostic and repair events for drone airframes with validated repair dates and positive service costs.
- Grounds airframes when logging BAD status and restores flight readiness upon recording a subsequent GOOD clearance.

Modules & Explanations - Database Triggers

AMRITA VISHWA VIDYAPEETHAM

1. trg_check_mission_payload_assignment

(BEFORE INSERT / UPDATE ON MISSIONS)

- Enforces that cargo weight cannot exceed drone payload capacity (PACKAGES.WEIGHT <= DRONES.PAYLOAD_CAP).
- Rejects insertion if the assigned drone, pilot, or package is already reserved by another active mission.

2. trg_check_mission_transition

(BEFORE UPDATE ON MISSIONS)

- Enforces valid lifecycle progression (IDLE → ONGOING/CANCELLED and ONGOING → OVER).
- Prohibits backward transitions, skipped stages, or tampering with terminal records (OVER and CANCELLED).

3. trg_check_charging_drone_not_flying

(BEFORE INSERT ON CHARGING_SESSIONS)

- Inspects active mission records and blocks charging session initiation for drones actively flying in transit.
- Prevents ground charging hardware hazards while airframes are operating in commercial airspace.

4. trg_check_mission_start_drone_not_charging

(BEFORE UPDATE ON MISSIONS)

- Intercepts mission launch attempts and blocks transition to ONGOING if any attached battery is actively charging.
- Ensures drones cannot be launched until physical charging cords and charging sessions are safely terminated.

5. trg_check_payment_mission_status

(BEFORE INSERT ON PAYMENTS)

- Restricts payment logging strictly to completed missions (MISSION_STATUS = 'OVER'), rejecting premature billing.
- Enforces billing idempotency by rejecting duplicate COMPLETED payment records for the same delivery.

6. trg_check_maintenance_validity

(BEFORE INSERT ON MAINTENANCE)

- Enforces database integrity constraints requiring repair and inspection costs to be strictly positive (COST > 0).
- Validates inspection timestamps, rejecting anomalous service records with future-dated repair entries.

Modules & Explanations - Transactions

AMRITA VISHWA VIDYAPEETHAM

1. Atomic Dispatch & Launch Workflow

- Employs pessimistic row-level locking (SELECT ... FOR UPDATE) on missions and pilots to prevent double-dispatch races.
- Synchronizes status transitions across MISSIONS (ONGOING) and PILOTS (ON_MISSION), rolling back on any conflict.

2. Delivery Finalisation & State Sync Workflow

- Executes an atomic multi-table commit across MISSIONS, PILOTS, DRONES, and PACKAGES upon delivery arrival.
- Guarantees all updates succeed together: mission completed, pilot freed, airframe relocated, and parcel closed.

3. Billing Settlement & Idempotency Workflow

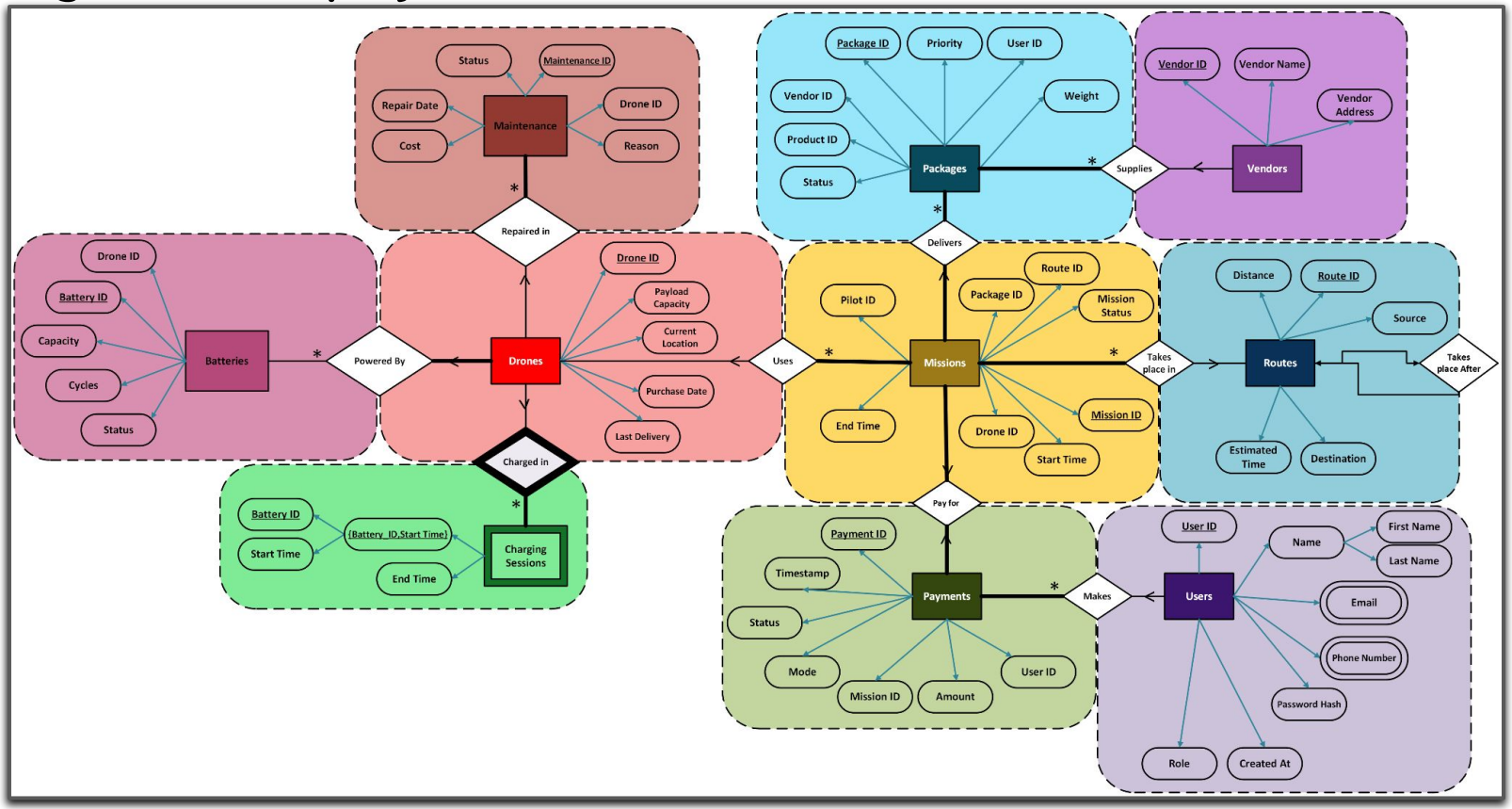
- Encapsulates payment validation, amount derivation, and duplicate checking within an isolated transaction.
- Rolls back immediately if an uncompleted delivery is billed or if a duplicate payment attempt is detected.

4. Structured SQL Exception & Rollback Handling

- All critical database procedures define explicit exit handlers (DECLARE EXIT HANDLER FOR SQLEXCEPTION).
- Ensures that unexpected query failures, network disconnections, or trigger aborts trigger a clean ROLLBACK.

Database Design Overview:

ER Diagram of the project:



ER-Diagram - Relationship Analysis:

1. **Makes**
   - 1.1. **Entities:** User → Payment
   - 1.2. **Relationship Type:** Binary
   - 1.3. **Cardinality:** One-to-Many (1:N)
   - 1.4. **Interpretation**
     - 1.4.1. One user can make multiple payments.
     - 1.4.2. Every payment belongs to one user.
   - 1.5. **Participation**
     - 1.5.1. User → Partial
     - 1.5.2. Payment → Total
   - 1.6. **Primary Key**
     - 1.6.1. Payment ID
2. **Pay For**
   - 2.1. **Entities:** Payment → Package
   - 2.2. **Relationship Type:** Binary
   - 2.3. **Cardinality:** One-to-One (1:1)
   - 2.4. **Interpretation**
     - 2.4.1. One payment corresponds to one package.
     - 2.4.2. Each package has one associated payment.
   - 2.5. **Participation**
     - 2.5.1. Payment → Total
     - 2.5.2. Package → *Total (every delivery has to be paid for)*
   - 2.6. ***Primary Key***
     - *2.6.1. Payment ID*
     - *2.6.2. Mission ID*
3. **Uses**
   - 3.1. **Entities:** Mission → Drone
   - 3.2. **Relationship Type:** Binary
   - 3.3. **Cardinality:** Many-to-One (N:1)
   - 3.4. **Interpretation**
     - 3.4.1. Multiple **Missions** may utilise the same **Drone** over different delivery schedules.
     - 3.4.2. Each **Mission** is assigned one **Drone**.
   - 3.5. **Participation**
     - 3.5.1. Mission → Total
     - 3.5.2. Drone → Partial
   - 3.6. **Primary Key**
     - 3.6.1. Drone ID
4. **Powered By**
   - 4.1. **Entities:** Drone → Battery
   - 4.2. **Relationship Type:** Binary
   - 4.3. **Cardinality:** One-to-Many (1:N)
   - 4.4. **Interpretation**
     - 4.4.1. One Drone operates using multiple Batteries.
     - 4.4.2. One Battery is assigned to one Drone at a given time.
   - 4.5. **Participation**
     - 4.5.1. Drone → Total
     - 4.5.2. Battery → Partial
   - 4.6. **Primary Key**
     - 4.6.1. Battery ID

ER-Diagram - Relationship Analysis:

AMRITA VISHWA VIDYAPEETHAM

5. **Charged In**
   - 5.1. **Entities:** Battery → Charging Session
   - 5.2. **Relationship Type:** Binary
   - 5.3. **Cardinality:** One-to-Many (1:N)
   - 5.4. **Interpretation**
     - 5.4.1. A battery may undergo multiple charging sessions.
     - 5.4.2. Every charging session corresponds to one battery.
   - 5.5. **Participation**
     - 5.5.1. Battery → Partial
     - 5.5.2. Charging Session → Total
   - 5.6. **Primary Key**
     - 5.6.1. Battery ID
     - 5.6.2. Charge Time
     - 5.6.3. Drone ID
6. **Repaired In**
   - 6.1. **Entities:** Drones → Maintenance
   - 6.2. **Relationship Type:** Binary
   - 6.3. **Cardinality:** One-to-Many (1:N)
   - 6.4. **Interpretation**
     - 6.4.1. One drone may require multiple maintenance..
     - 6.4.2. Every maintenance record is associated with one drone.
   - 6.5. **Participation**
     - 6.5.1. Drone → Partial
     - 6.5.2. Maintenance → Total
   - 6.6. **Primary Key**
     - 6.6.1. Maintenance ID
7. **Delivers**
   - 7.1. **Entities:** Mission → Package
   - 7.2. **Relationship Type:** Binary
   - 7.3. **Cardinality:** One-to-Many (1:N)
   - 7.4. **Interpretation**
     - 7.4.1. One Mission may require multiple Packages
     - 7.4.2. Each Package belongs to one Mission
   - 7.5. **Participation**
     - 7.5.1. Mission → Total
     - 7.5.2. Package → Total
   - 7.6. **Primary Key**
     - 7.6.1. Package ID
8. **Supplies**
   - 8.1. **Entities:** Vendor → Package
   - 8.2. **Relationship Type:** Binary
   - 8.3. **Cardinality:** One-to-Many (1:N)
   - 8.4. **Interpretation**
     - 8.4.1. A vendor may supply multiple packages.
     - 8.4.2. Every package originates from one vendor.
   - 8.5. **Participation**
     - 8.5.1. Vendor → Partial
     - 8.5.2. Package → Total
   - 8.6. **Primary Key**
     - 8.6.1. Vendor ID
     - 8.6.2. Package ID

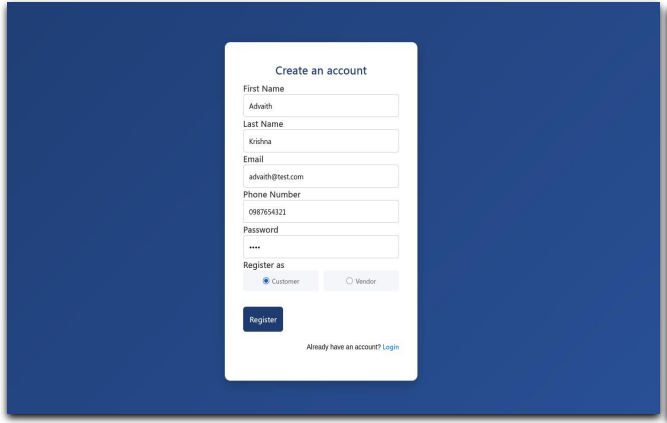
ER-Diagram - Relationship Analysis:

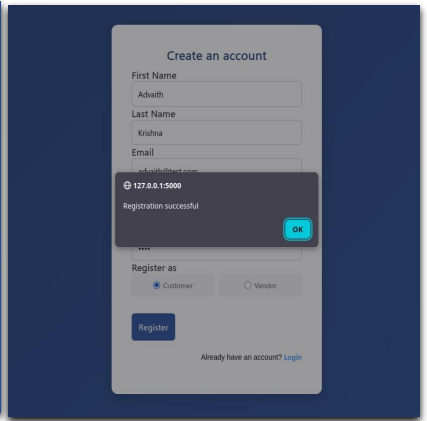
AMRITA VISHWA VIDYAPEETHAM

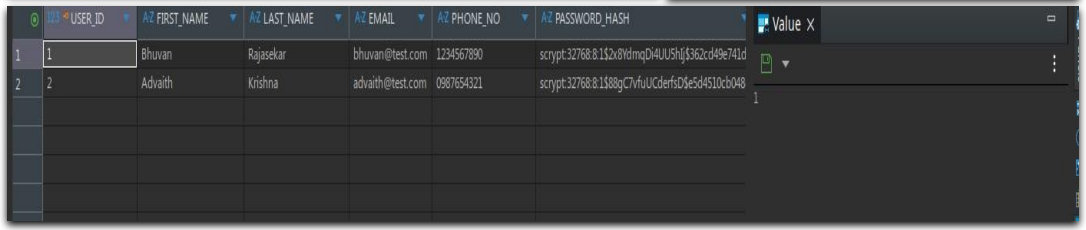
9. **Takes Place In**
   - 9.1. **Entities:** Mission → Route
   - 9.2. **Relationship Type:** Binary
   - 9.3. **Cardinality:** Many-to-One (N:1)
   - 9.4. **Interpretation**
     - 9.4.1. Multiple **Missions** may follow the same predefined **Route**.
     - 9.4.2. Each **Mission** is assigned one specific **Route**.
   - 9.5. **Participation**
     - 9.5.1. **Mission ->** Total
     - 9.5.2. **Route ->** Partial
   - 9.6. **Primary Key**
     - 9.6.1. Routes ID
10. **Takes Place After**
   - 10.1. **Entities:** Routes→ Routes
   - 10.2. **Relationship Type:** Unary
   - 10.3. **Cardinality:** One-to-One (1:N)
   - 10.4. **Interpretation**
     - 10.4.1. The Drone goes from Vendor to User, then the drone goes back from User to Vendor
     - 10.4.2. This both should happen sequentially only
   - 10.5. **Participation**
     - 10.5.1. Route → Partial
     - 10.5.2. Not every Route needs prior route
   - 10.6. **Primary Key**
     - 10.6.1. Routes ID

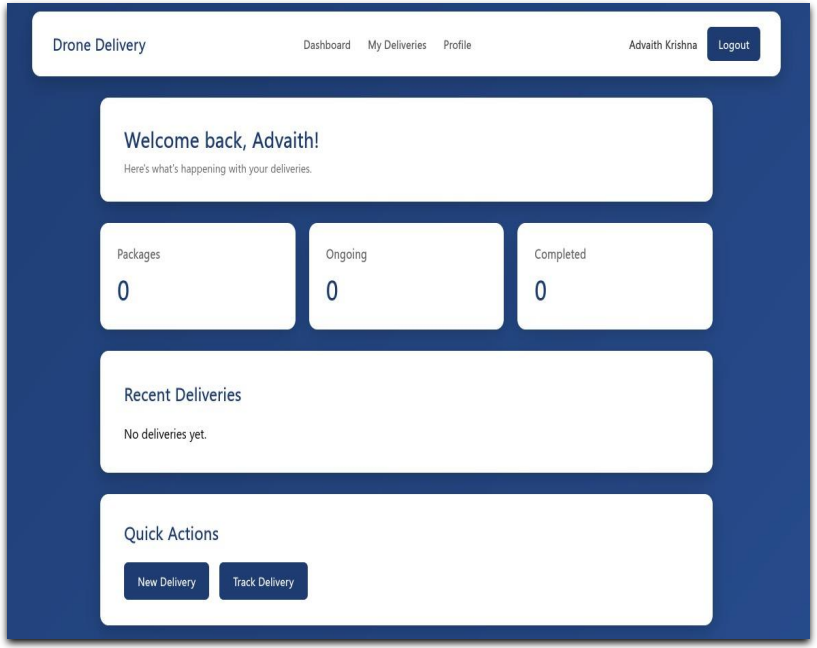
Results and Discussions

Create **R**ead **U** **D**





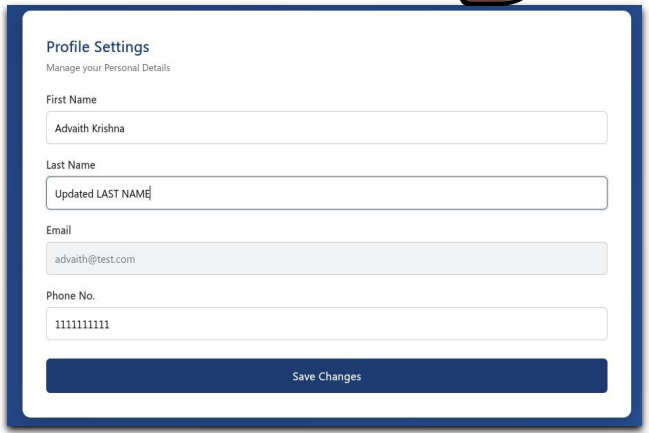




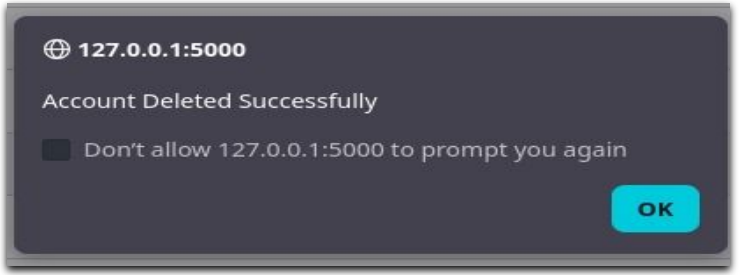
- **CREATE:** Registers a new user and inserts validated details into the USERS table.
- **READ:** Retrieves stored user information for authentication, dashboard access, and profile display.
- **DBMS Concepts:** INSERT, SELECT, data validation, and UNIQUE constraints.

Results and Discussions

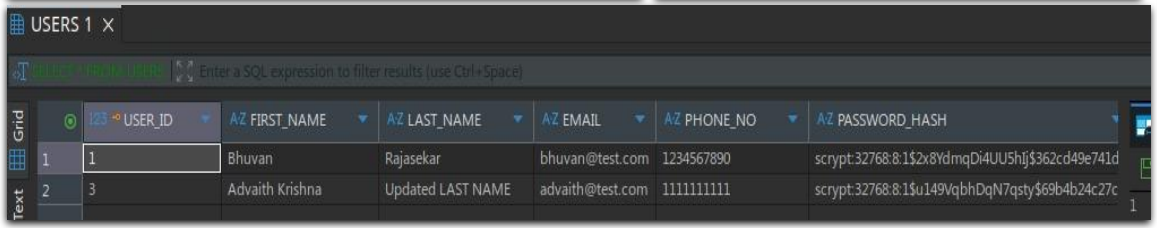
CRUpdate Delete











- Allows authenticated users to modify their profile information and save changes to the database.
- **DELETE:** Permanently removes a user account after confirmation.
- **DBMS Concepts:** UPDATE, DELETE, record modification, and data consistency.

Project Highlights

- Unified database for managing multiple drone delivery operations.
- Relational data model connecting users, packages, drones, missions, and payments.
- Primary and Foreign Key relationships for maintaining referential integrity.
- Web interface for database interaction and record management.

Comparison with Existing Approaches

Conventional Approach	Proposed System
Information may be managed separately	Centralised relational database
Manual coordination between records	Linked entities using foreign keys
Difficult to maintain consistent information	Structured data and constraints
Multiple independent management tasks	Integrated management platform

Impact: Provides a structured foundation for efficient and scalable drone delivery management.

Advantages & Limitations

Advantages

- Centralised storage of delivery information.
- Reduced data redundancy and improved consistency.
- Easy data retrieval and modification through CRUD operations.
- Automated enforcement of business rules and flight safety invariants via database triggers.
- Guaranteed data integrity across multi-table operations through ACID-compliant transactions.
- Prevention of race conditions and resource double-booking using row-level concurrency control.
- Enhanced data security and operational privacy through role-based access control and tenant isolation.

Limitations

- The system requires further testing for large-scale deployment.
- Real-world autonomous drone operation is not demonstrated.
- Advanced concurrency control and performance optimization are not yet validated.
- Telemetry tracking relies on discrete waypoint names rather than live, real-time GPS coordinate feeds.
- Payment processing operates in a simulated demo environment without integration to commercial banking gateways.

Challenges & Learnings

Challenge	Learning
1. Multi-table state synchronisation during failures	Implemented ACID transactions with automated rollback to prevent partial updates.
2. Concurrent resource allocation causes double booking	Applied row level locking using SELECT FOR UPDATE to serialise access.
3. Application validation easily bypassed by users	Enforced payload constraints and status transitions using database level triggers.
4. Deleting records violates foreign key constraints	Designed non-destructive status workflows to preserve historical relational audit logs.
5. Distinguishing current condition from historical logs	Structured temporal subqueries to fetch latest maintenance record for readiness.

Budgeting

Thank You

Introduction:

- The rapid growth of autonomous logistics has increased the demand for efficient systems capable of managing delivery operations and large volumes of interconnected data.
- A Drone Delivery Management System integrates operational information related to:
  - Users
  - Drones
  - Packages
  - Batteries
  - Delivery Routes
  - Maintenance Records
  - Payments Delivery Missions
- A well-designed relational database enables these components to interact efficiently while maintaining **accuracy, consistency and data integrity** throughout the system.



Abstract :

The **Drone Delivery Management System** is designed to support the efficient management of autonomous delivery operations through a structured relational database. The system maintains information related to users, drones, batteries, packages, routes, missions, vendors, maintenance records and payments, ensuring that all operational data remains organised and consistent. The current implementation establishes the database design and supports complete CRUD operations. Future development will extend the system by incorporating advanced database concepts such as transaction management, concurrency control and optimisation techniques to improve reliability and scalability

Objectives:

- To design a robust relational database for managing drone delivery operations.
- To minimise data redundancy while maintaining data consistency and integrity.
- To establish meaningful relationships between all major system entities.
- To implement secure and efficient CRUD operations for database management.
- To develop an intuitive interface for interacting with the database.
- To provide a scalable foundation for future integration of advanced DBMS concepts.

Key Features:

AMRITA VISHWA VIDYAPEETHAM

Operation Features:

- User Registration and Management
- Package Booking and Tracking
- Drone Allocation
- Route Management
- Battery Monitoring
- Maintenance Scheduling
- Vendor Management
- Payment Processing

Database Features:

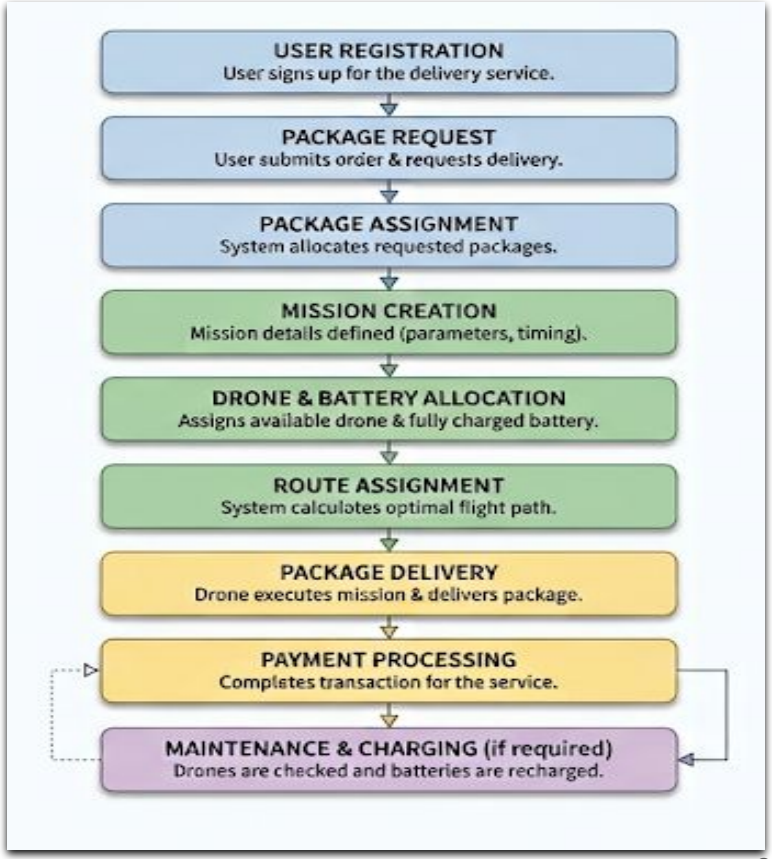
- *Structured Relational Database*
- *Primary and Foreign Key Constraints*
- *Reduced Data Redundancy*
- *Data Integrity Enforcement*
- *Complete CRUD Operations*
- *Extensible Database Design*

Design Overview:

The database design has been built around focusing on the following areas:

- Identifying major system entities.
- Defining their attributes.
- Establishing meaningful relationships.
- Maintaining data integrity through keys and constraints.
- Minimising redundancy while ensuring scalability.

Overall Workflow of the Project



ER-Diagram - Entities and Attributes:

AMRITA VISHWA VIDYAPEETHAM

User Table:

```
CREATE TABLE IF NOT EXISTS USERS(
        USER_ID INT PRIMARY KEY AUTO_INCREMENT,
        FIRST_NAME VARCHAR(25) NOT NULL,
        LAST_NAME VARCHAR(25) NOT NULL,
        EMAIL VARCHAR(50) UNIQUE NOT NULL,
        PHONE_NO VARCHAR(10) UNIQUE NOT NULL,
        PASSWORD_HASH VARCHAR(255) NOT NULL,
        CREATED_AT DATE NOT NULL,
        ROLE ENUM("VENDOR", "CUSTOMER") NOT NULL,

        CHECK(PHONE_NO REGEXP '^[0-9]{10}$'),
        CHECK(EMAIL REGEXP '^[A-Za-z0-9_]+@[A-Za-z]+\\.[A-Za-z]{2,}$')
    );
```

- Strong entity storing customer and vendor details; uniquely identified by USER_ID with role-based specialization.

ER-Diagram - Entities and Attributes:

Vendor Table:

```
1   CREATE TABLE IF NOT EXISTS VENDORS(
2           VENDOR_ID INT PRIMARY KEY,
3           VENDOR_NAME VARCHAR(50) NOT NULL,
4           VENDOR_ADDRESS VARCHAR(100) NOT NULL
5       );
```

- Strong entity representing vendors; independently identified and referenced by packages through foreign-key relationships.

ER-Diagram - Entities and Attributes:

Drones Table:

```
CREATE TABLE IF NOT EXISTS DRONES(
        DRONE_ID INT PRIMARY KEY,
        PAYLOAD_CAP INT NOT NULL,
        CURR_LOCATION VARCHAR(100) NOT NULL,
        PURCHASE_DATE DATE NOT NULL,
        LAST_DELIVERY DATETIME NOT NULL
    );
```

- Strong entity representing delivery drones, identified by DRONE_ID and participating in multiple operational relationships.

ER-Diagram - Entities and Attributes:

AMRITA VISHWA VIDYAPEETHAM

श्रद्धावान् लभते ज्ञानम्

Packages Table:

```
CREATE TABLE IF NOT EXISTS PACKAGES(
        PACKAGE_ID INT PRIMARY KEY,
        USER_ID INT NOT NULL,
        VENDOR_ID INT NOT NULL,
        PRODUCT_ID INT NOT NULL,
        WEIGHT INT NOT NULL,
        PRIORITY INT NOT NULL,
        STATUS ENUM("AVAILABLE" , "NOT AVAILABLE") NOT NULL DEFAULT "AVAILABLE",

        FOREIGN KEY(USER_ID) REFERENCES USERS(USER_ID),
        FOREIGN KEY(VENDOR_ID) REFERENCES VENDORS(VENDOR_ID),

        UNIQUE(VENDOR_ID, PRODUCT_ID)
    );
```

- Strong entity representing deliverable packages; connects customers and vendors through foreign-key relationships.

ER-Diagram - Entities and Attributes:

Routes Table:

```
CREATE TABLE IF NOT EXISTS ROUTES(
        ROUTE_ID INT NOT NULL,
        SOURCE VARCHAR(25) NOT NULL,
        DESTINATION VARCHAR(25) NOT NULL,
        DISTANCE DECIMAL(10,2) NOT NULL,
        ESTIMATED_TIME INT NOT NULL
    );

ALTER TABLE ROUTES
    ADD CONSTRAINT MISSION_ID INT NOT NULL
    FOREIGN KEY(MISSION_ID) REFERENCES MISSIONS(MISSION_ID);
```

- Strong entity defining delivery paths, including source, destination, distance, and estimated travel time.

ER-Diagram - Entities and Attributes:

AMRITA VISHWA VIDYAPEETHAM

Missions Table:

```
CREATE TABLE IF NOT EXISTS MISSIONS(
        MISSION_ID INT PRIMARY KEY,
        DRONE_ID INT NOT NULL,
        PILOT_ID INT NOT NULL,
        PACKAGE_ID INT NOT NULL,
        ROUTE_ID INT NOT NULL,
        MISSION_STATUS ENUM("IDLE", "ONGOING", "OVER") NOT NULL,
        START_TIME DATETIME NOT NULL,
        END_TIME DATETIME NOT NULL,

        FOREIGN KEY(DRONE_ID) REFERENCES DRONES(DRONE_ID),
        FOREIGN KEY(PACKAGE_ID) REFERENCES PACKAGES(PACKAGE_ID)
    );
```

- Strong entity connecting drones, packages, pilots, and routes to represent delivery operations.

ER-Diagram - Entities and Attributes:

Payments Table:

```
CREATE TABLE IF NOT EXISTS PAYMENTS(
        PAYMENT_ID INT PRIMARY KEY,
        USER_ID INT NOT NULL,
        MISSION_ID INT NOT NULL,
        TIMESTAMP DATETIME NOT NULL,
        MODE ENUM('COD' , 'ONLINE') NOT NULL DEFAULT "ONLINE",
        AMOUNT INT NOT NULL,
        STATUS ENUM("COMPLETED", "PROCESSING", "FAILED") NOT NULL,

        FOREIGN KEY(USER_ID) REFERENCES USERS(USER_ID),
        FOREIGN KEY(MISSION_ID) REFERENCES MISSIONS(MISSION_ID),

        CHECK(AMOUNT>0)
    );
```

- Strong entity recording payment transactions associated with users and completed delivery missions.

ER-Diagram - Entities and Attributes:

AMRITA VISHWA VIDYAPEETHAM

Batteries Table:

```
CREATE TABLE IF NOT EXISTS BATTERIES(
        BATTERY_ID INT PRIMARY KEY,
        DRONE_ID INT NOT NULL,
        CAPACITY INT NOT NULL,
        CYCLES INT NOT NULL,
        STATUS ENUM('GOOD' , 'BAD') NOT NULL DEFAULT "GOOD",

        FOREIGN KEY(DRONE_ID) REFERENCES DRONES(DRONE_ID)

    );
```

- Strong entity where each battery is associated with one drone through identifying ownership.

ER-Diagram - Entities and Attributes:

AMRITA VISHWA VIDYAPEETHAM

Maintenance Table:

```
CREATE TABLE IF NOT EXISTS MAINTENANCE(
        MAINTENANCE_ID INT PRIMARY KEY,
        DRONE_ID INT NOT NULL,
        STATUS ENUM('GOOD' , 'BAD') NOT NULL DEFAULT 'GOOD',
        REPAIR_DATE DATE NOT NULL,
        COST INT NOT NULL,
        REASON VARCHAR(100) NOT NULL DEFAULT 'IN GOOD CONDITION',

        FOREIGN KEY(DRONE_ID) REFERENCES DRONES(DRONE_ID),

        CHECK(COST>0)
    );
```

- Strong entity recording drone maintenance; each record depends on its associated drone through a foreign-key relationship.

ER-Diagram - Entities and Attributes:

AMRITA VISHWA VIDYAPEETHAM

Charging Sessions Table:

```
CREATE TABLE IF NOT EXISTS CHARGING_SESSIONS(
        BATTERY_ID INT NOT NULL,
        START_TIME DATETIME NOT NULL,
        END_TIME DATETIME NOT NULL,

        PRIMARY KEY(BATTERY_ID, START_TIME),

        FOREIGN KEY(BATTERY_ID) REFERENCES BATTERIES(BATTERY_ID)
    );
```

- Weak entity associated with batteries; composite key identifies each charging session using battery and start time.

ER-Diagram - Entities and Attributes:

AMRITA VISHWA VIDYAPEETHAM

Charging Sessions Table:

```
CREATE TABLE IF NOT EXISTS CHARGING_SESSIONS(
        BATTERY_ID INT NOT NULL,
        START_TIME DATETIME NOT NULL,
        END_TIME DATETIME NOT NULL,

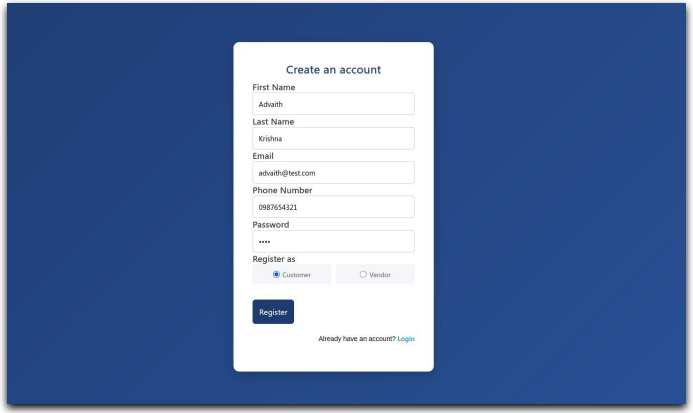
        PRIMARY KEY(BATTERY_ID, START_TIME),

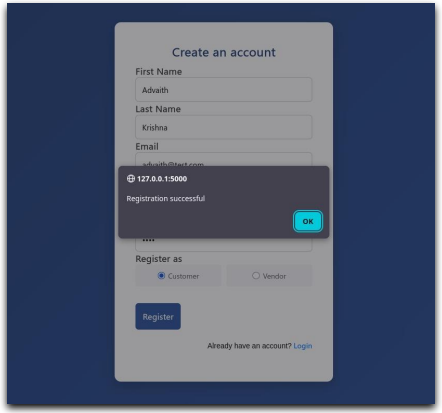
        FOREIGN KEY(BATTERY_ID) REFERENCES BATTERIES(BATTERY_ID)
    );
```

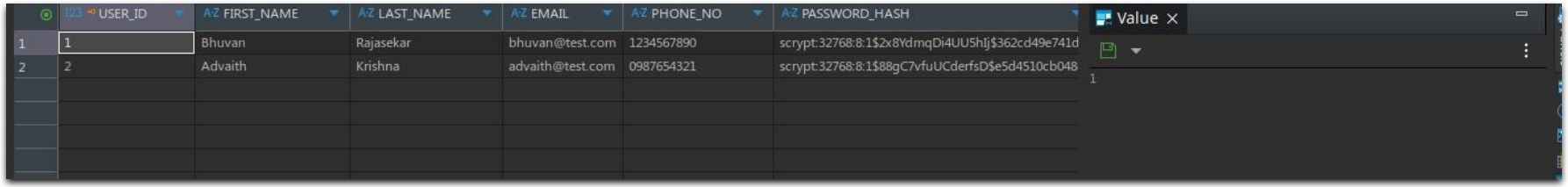
- Weak entity associated with batteries; composite key identifies each charging session using battery and start time.

Demo Screenshots

Create RUD







	USER_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NO	PASSWORD_HASH
1	1	Bhuvan	Rajasekar	bhuvan@test.com	1234567890	scrypt:32768:8:1$2x8YdmqDi4UU5hIj$362cd49e741d
2	2	Advaith	Krishna	advaith@test.com	0987654321	scrypt:32768:8:1$88gC7vfuUCderfsD$e5d4510cb048

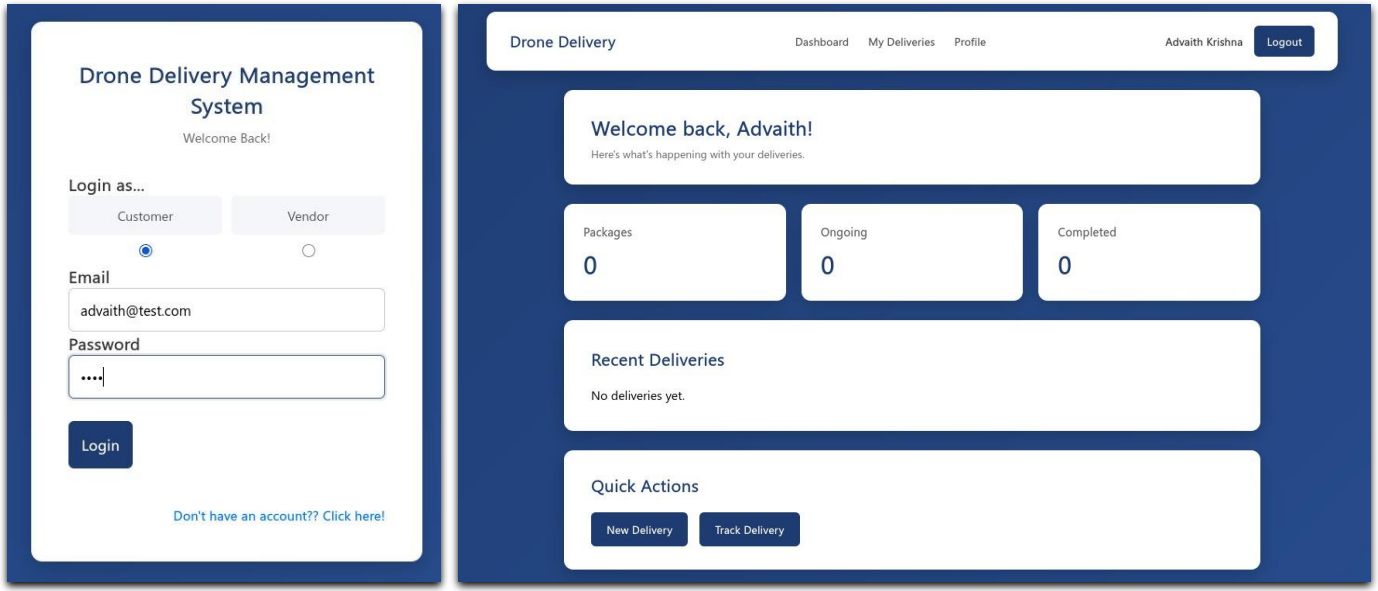
Value

1

- Registers new users with validated details and securely hashed passwords.

Demo Screenshots

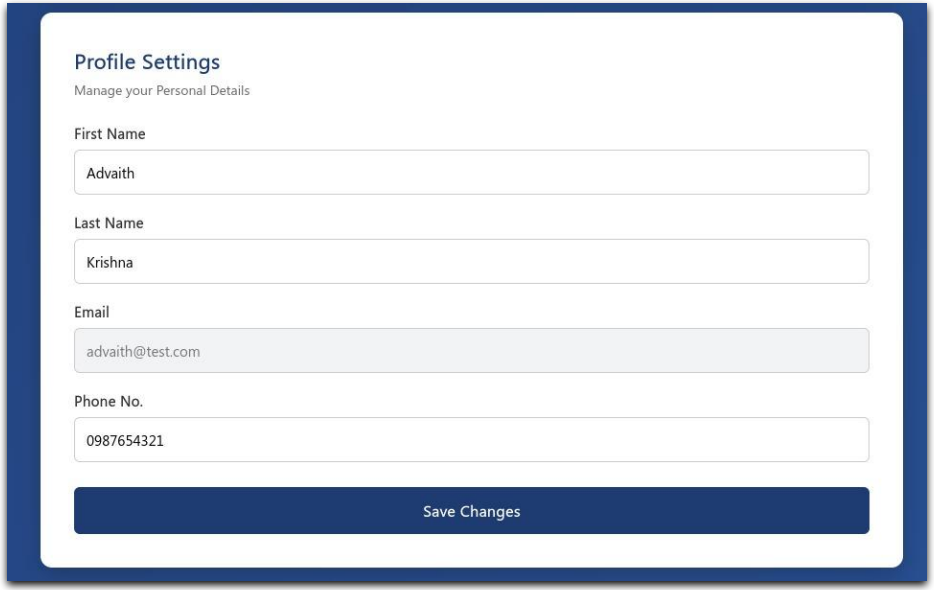
CRead UD



- Retrieves user profiles and authenticates users using email and role.

Demo Screenshots

CRead UD

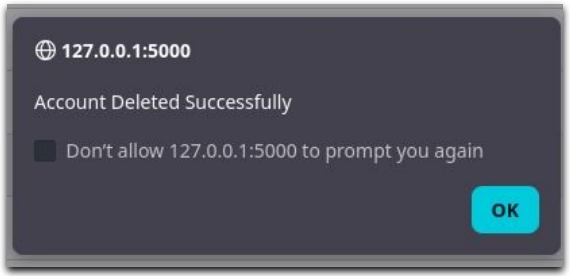


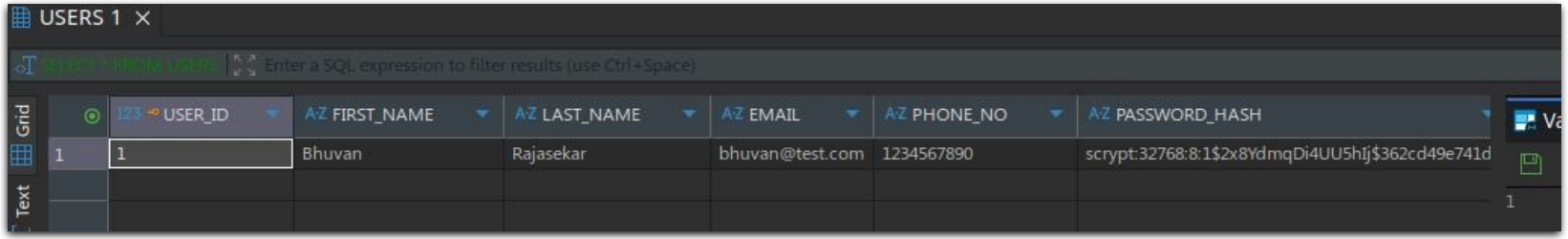
- Displays User's profile details in input boxes with the option to edit it and store it in the db

Demo Screenshots

CRUDelete







- Permanently removes the authenticated user account from the database

Pending Works

AMRITA VISHWA VIDYAPEETHAM

- Read and Create data for PACKAGES, MISSIONS and ROUTES
- Route the Completed DATA
- Add Vendor Authentication to Store them @ VENDORS
- CREATE DRONES and related tables